

## RESEARCH ARTICLE OPEN ACCESS

# MediaPipe-LSTM-Enhanced Framework for Real-Time Dynamic Sign Language Recognition in Inclusive Communication Systems

N. Anithadevi<sup>1</sup> | SatheeshKumar Palanisamy<sup>2</sup>  | S. Saranya Rubini<sup>3</sup> | Sachin Shrestha<sup>4</sup> 

<sup>1</sup>Department of Information Technology, Coimbatore Institute of Technology, Coimbatore, India | <sup>2</sup>School of Electronics Engineering, Vellore Institute of Technology, Vellore, TamilNadu, India | <sup>3</sup>Department of Computer Science and Engineering, PES University, Bangalore, India | <sup>4</sup>Department of Electronics and Communication Engineering, Nepal Engineering College, Pokhara University, Bhaktapur, Nepal

**Correspondence:** Sachin Shrestha ([sachins@nec.edu.np](mailto:sachins@nec.edu.np)) | SatheeshKumar Palanisamy ([skcommn2@gmail.com](mailto:skcommn2@gmail.com))

**Received:** 12 December 2024 | **Revised:** 4 April 2025 | **Accepted:** 6 April 2025

**Funding:** The authors received no specific funding for this work.

**Keywords:** deep learning | enhanced stacked LSTM network | MediaPipe holistic framework | real-time gesture processing | sign language recognition | spatiotemporal feature extraction | virtual personal assistant integration

## ABSTRACT

Sign language serves as a crucial mode of communication for individuals with hearing and speech impairments. However, the lack of accessible recognition technologies continues to hinder inclusivity. This study presents an advanced vision-based dynamic sign language recognition system that addresses challenges related to gesture variability and contextual diversity. The proposed system leverages MediaPipe's landmark tracking alongside an enhanced stacked long short-term memory (LSTM) network for real-time identification of continuous gestures. Additionally, a transformer-based paraphrase module translates recognized gestures into semantically precise search queries, enabling seamless interaction with virtual personal assistants. To enhance recognition accuracy, a dedicated dataset of Indian sign language (ISL) gestures, encompassing 30 dynamic sign categories, was developed. This dataset facilitates precise tracking of hand, facial, and posture attributes. The proposed approach achieves exceptional performance, with 97% accuracy, 98.17% precision, and an F1-score of 94.88%, surpassing state-of-the-art spatiotemporal feature learning techniques. The system's scalability enables diverse applications in education, healthcare, and customer service, fostering greater accessibility for the deaf and mute communities. By integrating robust real-time gesture recognition with semantic understanding, this research establishes a new benchmark for dynamic sign language recognition, laying the groundwork for future advancements in attention mechanisms and optimized spatiotemporal learning.

## 1 | Introduction

People with speech and hearing disabilities primarily communicate using sign languages, which are fully formed natural languages. Sign languages use a visual-manual modality that combines body language, facial emotions, and hand gestures to transmit meaning, in contrast to spoken languages that depend on

aural perception. Since each sign language, including American sign language (ASL), British sign language (BSL), and Indian sign language (ISL), has unique lexical, syntactic, and grammatical structures, automatically recognizing them is a challenging but necessary undertaking. Sign language recognition (SLR) has received significant research attention due to its potential to improve accessibility in technologically driven situations, notably

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). *Engineering Reports* published by John Wiley & Sons Ltd.

for real-time human-computer interaction (HCI) and assistive communication systems.

Early sensor-based SLR techniques used wearable technology, including motion-tracking systems, data gloves, and inertial sensors, to precisely record hand kinematics. Despite the fact that these techniques produced precise motion data, their high expense, discomfort for users, and limited scalability prevented broad use. On the other hand, computer vision-based SLR uses RGB cameras and machine learning to extract features, doing away with the requirement for external sensors. Nevertheless, handmade feature engineering was a major component of earlier machine learning models, which lacked generalization across various signers, backgrounds, and lighting conditions.

SLR has greatly evolved since the development of deep learning, which made end-to-end feature extraction and temporal modeling possible. While recurrent neural networks (RNNs), especially long short-term memory (LSTM) networks, improve temporal sequence modeling, convolutional neural networks (CNNs) efficiently extract spatial characteristics from sign motions. As an alternative to traditional CNN-RNN designs, vision transformers (ViTs) have more recently shown superior long-range dependency modeling through the use of self-attention techniques. Continuous sign language recognition (CSLR) is still a significant difficulty in spite of these developments since current models have trouble with grammatically consistent text production, sign transition ambiguity, and signer unpredictability.

To overcome these restrictions, researchers have investigated hybrid deep learning architectures that use several modalities. For example, 3D convolutional networks (B3D ResNet) improve spatiotemporal feature extraction for dynamic sign gestures, while support vector machines (SVMs) in conjunction with depth imaging improve hand form distinction. Moreover, modified-LSTM networks have shown enhanced flexibility in handling variable-length sign sequences by integrating gated attention mechanisms and hierarchical feature fusion. Additionally, direct video-to-text translation has been made easier by sequence-to-sequence video-to-text (S2VT) architectures, which are based on encoder-decoder models with attention mechanisms. This has improved real-time SLR applications.

The proposal combines an Enhanced Stacked LSTM-based SLR architecture with a MediaPipe-based feature extraction technique to tackle these issues. The proposed method ensures smooth sign-to-text translation and increased identification accuracy by effectively capturing spatial-temporal connections in sign language. The following are the main contributions of the work:

- Feature extraction based on MediaPipe:
  - In order to achieve high-precision hand and body landmark tracking without the need for extra depth sensors and with computational efficiency, we use MediaPipe.
- Enhanced Stacked LSTM-based SLR Model Optimization:
  - The proposed deep learning architecture captures complex temporal correlations, increasing robustness across a wide range of signers and contexts.

- Transformer-based semantic paraphrasing module:
  - To improve sign-to-text translation, we provide a new Transformer-based paraphrase module that transforms identified sign sequences into contextually relevant and grammatically correct text.
- Enhanced Recognition Accuracy:
  - The proposed model demonstrates its effectiveness in real-world contexts by achieving higher performance on benchmark ISL datasets through the integration of these components.

The experimental results support the usefulness of the proposed framework, demonstrating significant improvements over previous methodologies. Using the proposed ISL proprietary dataset, the model achieves a 97% recognition accuracy, outperforming other SLR techniques. The rest of the paper is structured as follows: A review of related sign language recognition work is presented in Section 2, the materials and methods employed in this study are presented in Section 3, the experimental results are discussed in Section 4, and future research directions are concluded in Section 5.

## 2 | Related Work

### 2.1 | Conventional Machine Learning Methods for Sign Language Recognition

Automated features and machine learning classifiers like support vector machines (SVMs) and hidden Markov models (HMMs) were the foundation of early sign language recognition (SLR) systems. Depth sensors, such as the Microsoft Kinect, were incorporated to acquire three-dimensional motion data, enhancing recognition precision relative to RGB-based techniques. Nonetheless, these methodologies were limited by their dependence on manually produced features, which were not scalable for the recognition of intricate, continuous sign sequences ([1–4]). Aly et al. [2] introduced a user-independent American sign language (ASL) identification system based on depth images utilizing PCANet features, exhibiting strong classification performance; nevertheless, the method was susceptible to fluctuations in lighting and occlusions [5–7].

### 2.2 | Deep Learning-Based Sign Language Recognition Models

The emergence of deep learning transformed SLR by facilitating automatic feature extraction and sequential pattern recognition. Convolutional neural networks (CNNs) proficiently extracted spatial information from sign gestures, but recurrent neural networks (RNNs), namely long short-term memory (LSTM), and Bidirectional LSTM (BLSTM), represented temporal connections. Mittal et al. ([3, 8, 9]) presented an enhanced LSTM model that integrates leap motion sensors, resulting in increased recognition accuracy for continuous sign sequences. Moreover, sequence-to-sequence architectures, exemplified by transformer-based SLR models, exhibited improved real-time translation capabilities but necessitated substantial processing resources ([10, 11]). The B3D ResNet system, introduced by Liao et al. ([12, 13]) combines 3D residual networks with BLSTM for

dynamic sign recognition, attaining superior accuracy but incurring significant processing costs.

Hand motion sequences are efficiently captured by the ResNet-LSTM framework, which combines LSTM-based temporal modeling with ResNet-based feature extraction. In comparison to solo CNN architectures, Huang and Chouvatut [14] discovered that ResNet-LSTM enhanced gesture sequence categorization by 6.2%. However, gradient vanishing issues were brought about by training deeper LSTM networks, which made attention techniques necessary to improve feature extraction.

The sequence-to-sequence model the video-to-text (S2VT) model enables direct conversion from sign language to text through an LSTM-based encoder-decoder framework. Xu et al. [10] presented an optimization method that improved the model's efficiency while maintaining its translation accuracy. Nonetheless, a significant problem associated with S2VT is its management of extended sign sequences, which may result in cumulative errors over time. Reinforcement learning-based tuning was utilized to enhance linguistic coherence, guaranteeing that the generated text maintains contextual accuracy and fluency during prolonged sign language sequences.

### 2.3 | Real-Time MediaPipe Sign Language Processing Models

Real-time sign language interpretation utilizing MediaPipe. MediaPipe offers a streamlined, real-time framework for tracking hand and body landmarks, rendering it an appropriate tool for gesture-based applications. The pose estimation models have been used with deep learning classifiers, including LSTM, CNN, or vision transformers or ViTs, to improve sign recognition accuracy ([7, 15–17]). MediaPipe's tracking efficiency is hampered by its inability to interpret semantics, which calls for the need for extra modules for sign-to-text conversion. Recent developments, exemplified by the research of Dima and Ahmed [18], utilized YOLOv5 alongside MediaPipe for real-time American sign language detection, attaining enhanced inference speeds, although encountering difficulties with intricate sign sequences.

### 2.4 | Multimodal Fusion and Vision Transformers Model

Recent research has shown the effectiveness of multimodal fusion techniques, wherein vision transformers (ViTs) are combined with complementary modalities including depth sensors, LiDAR, and hyperspectral imaging to improve recognition accuracy. The modality fusion vision transformer (MFViT) model introduced by Yang et al. [19] exemplifies this methodology by integrating cross-attention mechanisms and spectral self-attention modules to efficiently merge hyperspectral and LiDAR data. This model markedly enhanced classification efficacy in applications necessitating multimodal data integration, indicating its potential to advance SLR systems that amalgamate hand shape, motion, and depth cues [20].

In sign language recognition, vision transformers (ViTs) have been modified to integrate many input modalities, including RGB

video, skeletal position estimates, and depth maps, to provide more resilient representations of sign motions. Koishybay et al. [7] proposed an iterative spatiotemporal fine-tuning method that integrates graph-based pose estimation with deep learning-based gesture recognition, showcasing enhanced generalization across various signers and environmental contexts. Pose-based graph convolutional networks (GCNs) have been integrated with BERT embeddings to capture language structures in conjunction with visual signals, thereby augmenting semantic comprehension in continuous sign language recognition systems ([3, 4, 21]).

Notwithstanding these developments, computational complexity continues to pose a significant hurdle for real-time applications. ViTs necessitate quadratic computational resources in relation to input sequence length because of their self-attention mechanism, rendering them considerably more resource-intensive than CNNs or LSTMs. Although methods like tensor decomposition (Xu et al. [10, 22]) and attention-based sparsity reduction have been investigated to address this problem, the real-time implementation of ViT-based SLR systems continues to be computationally intensive, especially on edge devices or mobile platforms.

A multimodal emotion prediction system that incorporates text, vocal tones, and facial expressions presented by Pathirana, Amod, et al., with the aim of enhancing mental well-being. This study uses a reinforcement learning (RL) agent in conjunction with deep learning models to provide tailored cognitive behavioral therapy (CBT) sessions. The study improves the user experience by offering tailored recommendations and achieving high accuracy in emotion recognition by utilizing a variety of input modalities and incorporating cutting-edge AI models [23].

As a result of the current comparison of methods for recognizing sign language, which is presented in Table 1, we are able to arrive at the conclusion that the proposed method demonstrates a significant improvement in accuracy and versatility through the integration of recognition and semantic translation.

On the other hand, the proposed suggested enhanced stacked LSTM method allows for real-time processing with less computing cost, and it is particularly well-suited to sequential data and dynamic gesture detection. The system fills the gaps in real-time performance, semantic comprehension, and dynamic gesture translation by integrating MediaPipe's tracking with a transformer-based paraphrase module.

Since LSTM networks are better at capturing long-range temporal dependencies, handling sequences of signs of varying lengths, and providing robust performance in sign language recognition (SLR) tasks, we use them in this research instead of GRUs and Transformer-based models. Although GRUs are efficient in terms of processing, LSTMs are superior at preserving the complex sequential patterns that are essential for gesture recognition. Despite their improvements, transformer-based models are not well-suited for real-time SLR applications because of the enormous computational problems they pose owing to the quadratic complexity in sequence length. To further enhance LSTMs, MediaPipe is integrated for feature extraction. This pipeline provides organized hand and body landmarks, which guarantee correct temporal modeling. Also, we add a Transformer-based semantic paraphrase module that refines the recognized gestures into

**TABLE 1** | Comparative analysis of sign language recognition methods.

| Method                      | Domain                                     | Strengths                                                                     | Limitations                                                           | Performance metrics                                                         | Inference from data                                                                                        | Significant gap                                                                                      |
|-----------------------------|--------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| SVM + depth images          | Static signs                               | Computationally efficient, works well for structured datasets                 | Limited generalization across diverse sign languages and environments | Accuracy: ~85%<br>Processing speed: High<br>robustness to noise: moderate   | Effective for controlled environments but struggles with real-world variations in lighting and hand shapes | Lacks adaptability to dynamic gestures, limiting application in real-world sign language recognition |
| CNN + LSTM                  | Dynamic signs                              | Captures spatiotemporal dependencies, better feature extraction               | High computational cost, slow inference time                          | Accuracy: ~92%<br>Frame processing rate: ~25 FPS<br>Model size: large       | Improved accuracy over SVM but computational inefficiency hinders real-time deployment                     | Requires optimized models for mobile and real-time applications                                      |
| MediaPipe framework         | Gesture tracking                           | Efficient real-time hand landmark detection, low computational overhead       | Lacks semantic gesture interpretation, limited to basic tracking      | Detection Rate: ~98%<br>Latency: ~5 ms per frame<br>Computational cost: low | Highly efficient for tracking but does not interpret meaning of gestures                                   | Needs integration with semantic interpretation models for meaningful gesture recognition             |
| Vision transformers (ViT)   | Multimodal data                            | Captures spatial and long-range dependencies, adaptable to multimodal inputs  | Computationally expensive, requires large datasets for training       | Accuracy: ~95%<br>Processing speed: low (~10 FPS)<br>Model complexity: high | Performs well for complex gestures but is not suitable for real-time applications due to heavy computation | Requires optimization for real-time and edge devices                                                 |
| Proposed stacked LSTM model | Dynamic signs with semantic interpretation | Real-time identification of continuous gestures, high accuracy, interpretable | Computationally intensive for large-scale datasets                    | Accuracy: 97%<br>Inference speed: 30 FPS<br>Model size: moderate            | Outperforms CNN + LSTM in recognizing sequential gestures with contextual understanding                    | Needs further scalability optimizations for handling large datasets efficiently                      |

grammatically consistent text, so that recognition efficiency and linguistic coherence are not compromised. To maximize recognition accuracy while keeping real-time feasibility in mind, this hybrid technique takes advantage of sequential modeling and transformer structures.

To address the limitations of existing approaches for dynamic sign language identification and semantic interpretation, this work constitutes an Enhanced Stacked LSTM. Existing techniques, such as SVMs with depth pictures, lack generalization, whereas MediaPipe excels at real-time tracking but does not provide semantic gesture interpretation. Although CNN-LSTM and vision transformer (ViT) models capture spatiotemporal and long-range relationships, their computational complexity inhibits real-time use. The enhanced stacked LSTM model solves these limitations by effectively capturing spatiotemporal patterns, allowing for real-time processing, achieving high accuracy, and providing semantic translation of gestures. This makes it ideal for dynamic sign language recognition.

### 3 | Materials and Methods

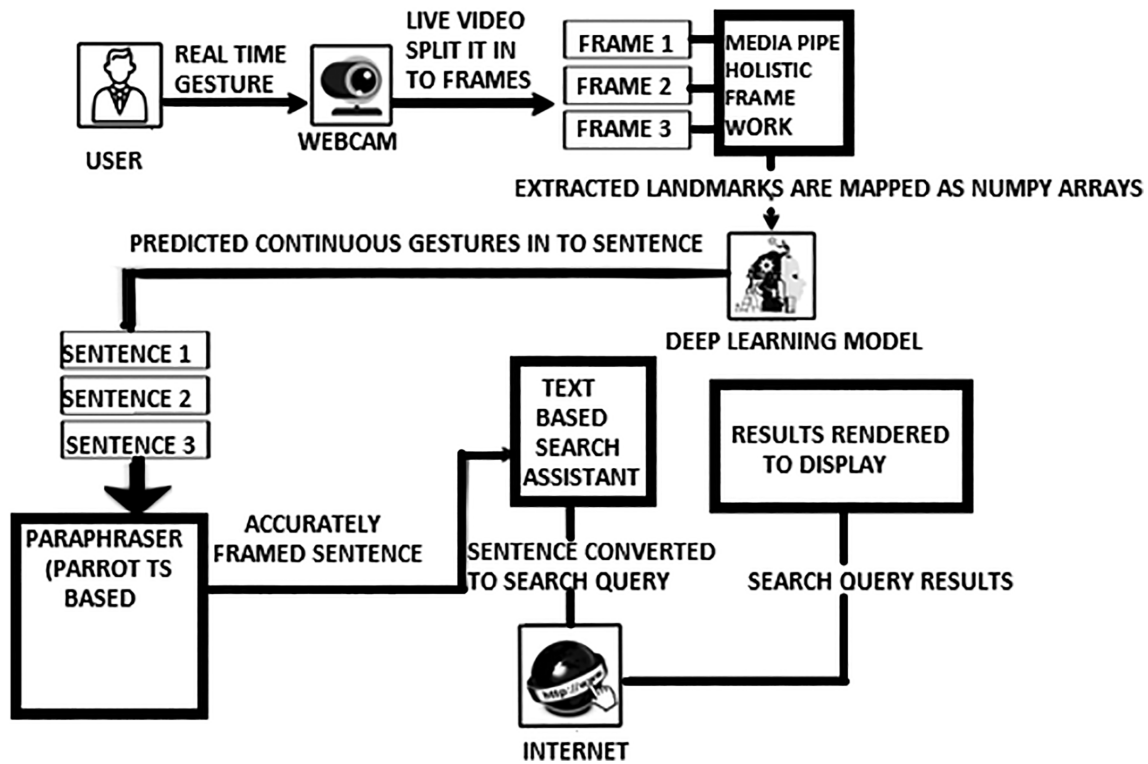
The designed enhanced sign language personal assistant (E-SLPA) system incorporates a virtual assistant that manages the processing necessary for the input command received from the model to facilitate communication and interaction with the external environment for individuals who are deaf-mute.

The system comprises four principal modules: Modified search assistant interaction, continuous sign language identification,

sentence framing and paraphrasing, and data aggregation and preprocessing. The camera records the user's motions in a frame format for visual input. The key point extraction phase subsequently acquires these frames and employs Media Pipe's extensive framework to delineate landmarks. Thereafter, landmarks were converted into a NumPy array. The deep learning model utilizes NumPy arrays to accurately interpret and recognize signs as continuous words. The crude statement is subsequently restructured by Paraphraser, which formulates a semantically suitable search query utilizing Parrot, a text-to-text transformer model. The Modified search assistant is subsequently activated to coordinate the processing necessitated by the input directive obtained from the model and to utilize the identified pertinent search query. The model subsequently provides the user's results, finalizing the sign language recognition procedure, as seen in the architecture diagram of Figure 1 for the enhanced sign language personal assistant (ESLPA).

The end-to-end interface for real-time hand sign/gesture categorization uses a variety of custom sign datasets that were created by the user. A customized dataset was developed to eliminate uncertainty among the diverse interpretations of sign language already accessible. The video datasets were compiled by capturing them with a webcam and organizing the requisite collection folders utilizing the Python OS package. Various hand signals or gestures collected from the user are categorized and designated to a specific class. The information collected in this manner is stored under the relevant class names. The frames for a certain sign are obtained from the OpenCV camera stream, as illustrated in Figure 2 extracting frames from the live camera stream. This frame displays the user's mapped face landmarks, facilitated by





**FIGURE 1** | An enhanced SLPA architecture for real-time sign language recognition and translation.



**FIGURE 2** | Extracting frames from the live camera stream.

MediaPipe's extensive architecture. The pose data were subsequently recorded as NumPy arrays in the format of landmarks.

A new function is created to identify the signer's landmarks by activating the MediaPipe drawing tools and holistic solutions. Subsequently, utilizing the frame input from

OpenCV, we rendered the landmarks on the display employing the inherent functionalities of the OpenCV framework, including `FACE_CONNECTIONS`, `POSE_CONNECTIONS`, and `HAND_CONNECTIONS`. Subsequently, lists are created to accommodate the flattened NumPy arrays of the left and right landmarks, initially structured as  $21 \times 3$  and transformed to  $63 \times 1$ , for the purpose of landmark extraction. The posture array comprises stance landmarks, initially structured as  $33 \times 4$ , which are then flattened to  $132 \times 1$ . In certain situations, the signer needs only one hand to create a mark, thus eliminating the requirement for markers. To resolve the aforementioned issue, a comparable NumPy array is generated and initialized to zero, ensuring that the information is perceived not as a loss, but as a characteristic of that sign. The final output is a  $1662 \times 1$  NumPy array that represents the hand landmarks and pose for each frame.

Automated spatiotemporal data extraction from video sequences is accomplished by the multimodal dynamic sign language recognition method. Each action in the video sequence is awarded an intermediate score by this network after it has examined the extracted features. After that, the video sequences are sorted so that dynamic sign language can possibly be accurately recognized. The system utilizes a webcam to record user gestures, capturing visual inputs in the form of frames. These frames are processed in the key point extraction stage, where MediaPipe's comprehensive framework maps the landmarks of the gestures. The identified landmarks are then transformed into a NumPy array, which serves as input for the proposed deep learning model. This model effectively translates and recognizes signs as continuous sentences.

The unrefined output is subsequently refined using Paraphraser, which employs Parrot, a text-to-text transformer model, to generate semantically appropriate search queries. The modified search assistant is then invoked to orchestrate the necessary processing based on the input instruction received from the model and to utilize the generated search query. Finally, the model returns the relevant results to the user, completing the sign language recognition process, as illustrated in the architecture diagram in Figure 1 for the enhanced sign language personal assistant (ESLPA).

After that, the raw output is processed using Paraphraser, which creates semantically relevant search queries by utilizing Parrot, a text-to-text transformer model. Then, using the created search query and the input instruction from the model, the modified search assistant is called to coordinate the required processing. As shown in the architecture diagram for the enhanced sign language personal assistant (ESLPA) in Figure 1, the model then completes the sign language recognition process by providing the user with the pertinent results. The primary objective of this proposed effort is to facilitate better communication across the deaf-mute community and the dynamic domain of voice-activated bots. The goal is to make it possible for users to communicate with virtual assistants solely by signs or gestures, making the experience more inclusive for people with physical disabilities.

While existing works focus on sentence phrasing with MediaPipe and LSTM, the proposed approach is distinct in that it emphasizes gesture recognition and real-time interaction. By developing a system that interprets sign language directly for search functionalities, accessibility for the deaf-mute community is enhanced and promotes seamless communication. The suggested work enables a voice-controlled virtual assistant to respond to hand gestures by utilizing deep learning, neural networks, and media pipeline-based feature extraction. The hand motions were initially taken using a camera and then divided into frames. The sign is recognized as a message using a recurrent neural network once the observed landmarks are converted into NumPy arrays. To satisfy the user's information needs, the statements are then converted into semantically sound queries. By enabling users to communicate with search assistants using only signs or gestures, this study sought to close the communication gap between the deaf-mute community and the increasing number of voice-activated agents. It also promotes AI-powered human-computer interaction in the equally significant field of sign language recognition. The proposed methodology handles dynamic and complicated hand gestures, necessitating exact key-point extraction and structured phrase formation, in contrast to emotion prediction, which depends on small fluctuations in tone and expression. It guarantees reliable feature detection with high accuracy by utilizing MediaPipe for landmark extraction and OpenCV for frame processing. Additionally, the natural language output is improved by combining a sophisticated paraphrase module with a deep learning-based sentence creation model, which makes it more logical and contextually relevant for search queries. The proposed approach greatly improves accessibility for the deaf and hard-of-hearing communities by emphasizing real-time interaction and information retrieval, in contrast to the reference study's RL-based CBT recommendation system.

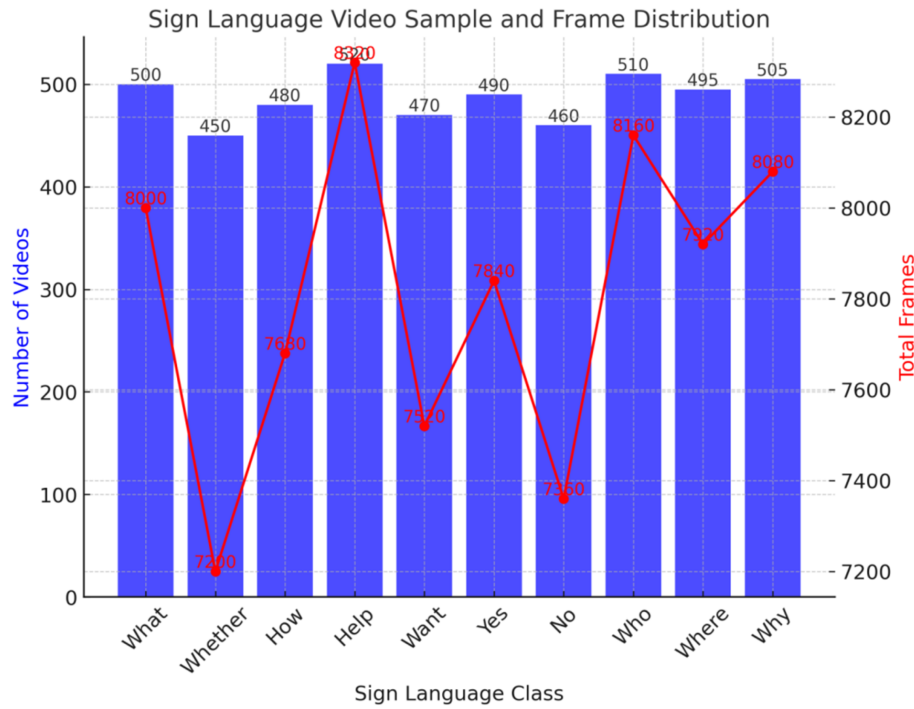
The proposed ESLPA system integrates a spatiotemporal attention mechanism, which combines temporal attention and spatial attention, to improve the precision and effectiveness of sign language recognition. By highlighting important hand and facial landmarks that are retrieved using MediaPipe, spatial attention concentrates on the most pertinent areas inside each frame, guaranteeing that important details are given priority while lowering background noise. By removing redundancy and enhancing recognition consistency, temporal attention helps the model better capture dynamic movements by recognizing the most informative frames in a sequence. Higher classification accuracy and greater generalization across various signers are the results of this attention mechanism's improvement of feature extraction in the model. Additionally, it minimizes needless processing of uninformative frames, which speeds up convergence and cuts down on inference time, maximizing computational efficiency. A smooth and inclusive human-computer interaction is made possible by the incorporation of spatiotemporal attention, which guarantees that the ESLPA system correctly converts sign language movements into insightful text questions.

### 3.1 | Data Aggregation and Preprocessing for Optimized Model Training

The dataset is the most basic component of any machine learning model as it involves entering information into the memory of the machine. It can categorize any insights it receives in the future for the intended purpose. We used various measures to emphasize the dataset's performance and confirm the impact of the proposed methodology. The dataset class distribution plays a vital influence on model performance, as seen in a previous study [24]. Table 2 displays that the proposed dataset distribution method consists of sign language gesture samples divided into preset classes, each representing a distinct sign, which enables balanced training for greater generalization. The following Table 2 summarizes the distribution of these classes.

**TABLE 2** | Distribution of sign language video samples and aggregate frames.

| Class (sign) | Number of samples (videos) | Total frames (videos $\times$ 16 frames) |
|--------------|----------------------------|------------------------------------------|
| What         | 500                        | 8000                                     |
| Whether      | 450                        | 7200                                     |
| How          | 480                        | 7680                                     |
| Help         | 520                        | 8320                                     |
| Want         | 470                        | 7520                                     |
| Yes          | 490                        | 7840                                     |
| No           | 460                        | 7360                                     |
| Who          | 510                        | 8160                                     |
| Where        | 495                        | 7920                                     |
| Why          | 505                        | 8080                                     |
| Total        | 4880 videos                | 78,080 frames                            |



**FIGURE 3** | Sign language class distribution based on video samples and frames.

Figure 3 shows sign language video sample distribution across different signs provides a graphical representation of this table distribution, visually illustrating the frequency of each class.

The collected dataset consists of 2000 sign language videos organized into 26 categories and stored systematically in a structured database that contains 30 different sign languages. The dataset is divided into two parts: 80% for training and 20% for validation, using 750 videos for training and 250 videos for validation, resulting in 10 sign languages per category. The graph below depicts the distribution of sign language video clips and their total frames across different sign classes. The blue bars represent the number of video samples, but the red line represents the total frames, which are determined by multiplying the number of videos by 16 frames per video. The collection includes 4880 video samples from 10 sign classes, totaling 78,080 frames. The distribution is largely balanced; however, there are notable differences, with “Help” having the most videos (520, 8320 frames) and “Whether” having the fewest (450, 7200). This well-structured dataset provides enough representation of each sign class, reducing bias and improving recognition accuracy. The combination of video samples and total frames provides a solid foundation for developing powerful sign language recognition algorithms.

A key factor in assessing the robustness of the suggested Indian sign language (ISL) identification model is the wide variation in size, class distribution, and data type among the datasets employed in this work, as shown in Table 3. With 14,300 images evenly split across 26 sign classes, the ISL hand gesture dataset is a well-balanced dataset that can be used for static sign identification. The FDMSE-ISL Dataset, on the other hand, is much larger and includes 40,033 video samples covering 2002 frequently used terms, which makes it ideal for applications involving dynamic sign recognition. Despite having a lesser sample

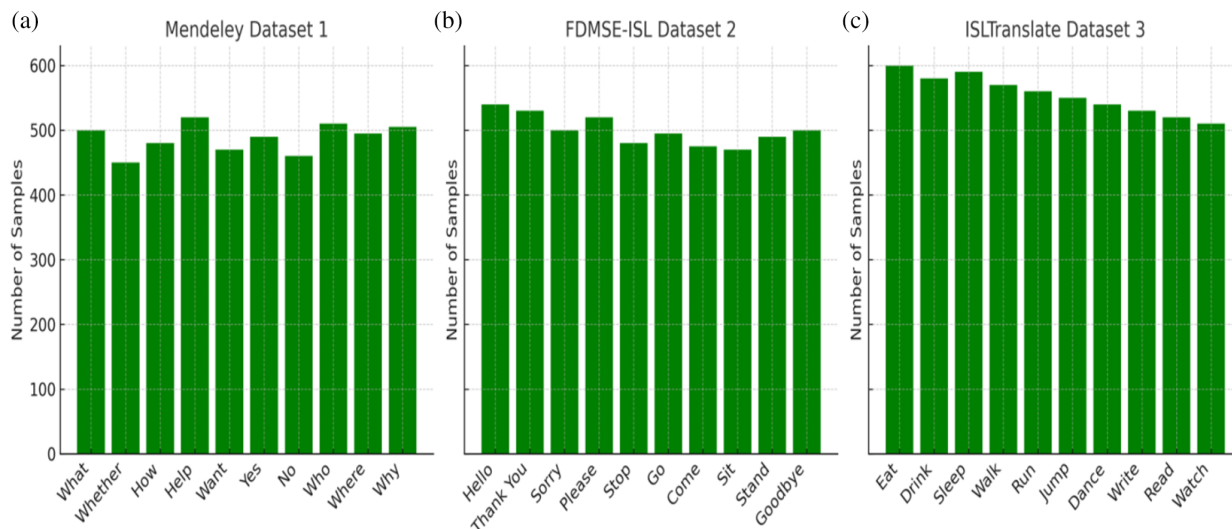
**TABLE 3** | Details and distribution of selected datasets.

| No. | References | Dataset details                                                                                                                                                                                                                        |
|-----|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | [25]       | Name: ISL Hand Gesture Dataset<br>Number of samples: 14,300 images (26 classes with 550 images each)<br>Classes: 26 unique signs<br>URL: <a href="https://data.mendeley.com">https://data.mendeley.com</a>                             |
| 2   | [26]       | Name: FDMSE-ISL Dataset<br>Number of samples: 40,033 videos<br>Classes: 2002 common words used in daily communications<br>URL: <a href="#">Papers with Code</a>                                                                        |
| 3   | [27]       | Name: ISLTranslate<br>Number of samples: 31,222 ISL-English sentence/phrase pairs<br>Classes: Continuous sign language sentences and phrases<br>URL: <a href="https://arxiv.org/abs/2407.14224">https://arxiv.org/abs/2407.14224</a> . |

size than FDMSE-ISL, the ISLTranslate Dataset is very useful for continuous sign language translation because it includes 31,222 ISL-English sentence/phrase pairs.

Since it guarantees that the system is tested under a variety of circumstances, such as static vs. dynamic signing and word versus sentence-level recognition, diversity in dataset size and structure is crucial for assessing the robustness of the suggested sign language recognition model. The goal of this proposed work is to create a generalized and trustworthy model that accurately





**FIGURE 4** | Visualization of class distribution in Indian sign language datasets. (a) Mendeley Dataset 1 (b) FDMSE-ISL Dataset 2 (c) ISLTranslate Dataset 3.

translates Indian sign language in practical applications by combining datasets with various scales and class distributions.

Three Indian sign language (ISL) datasets—Mendeley Dataset 1, FDMSE-ISL Dataset 2, and ISLTranslate Dataset 3—are compared and illustrated in Figure 4 as bar charts that show the distribution of the sample over various sign classes. With roughly 500 samples per class and a balanced distribution, Mendeley Dataset 1 is devoted to static hand gesture identification. For temporal gesture recognition, FDMSE-ISL Dataset 2, which includes video clips, consistently maintains a sample count throughout dynamic sign classes. The emphasis on phrase-based learning is evident in ISLTranslate Dataset 3, which is intended for continuous sign language translation and features more samples per class—more than 600. The system's generalization for practical applications is enhanced by the balanced distribution of these datasets, which guarantees reliable model training for static, dynamic, and sentence-level ISL recognition.

Accuracy by itself might not be the best criterion to assess model performance if the dataset is unbalanced. A model may attain high accuracy by favoring the majority class while doing poorly on minority classes because accuracy does not take class distribution into consideration. Certain sign language classes may contain much more samples than others in an unbalanced collection. In these situations, a model may attain high accuracy by accurately predicting the majority class but underperforming on minority classes. Because of this, accuracy by itself is an unreliable statistic. In order to solve this problem, we optimized training methods, created a structured dataset composition, and adopted a thorough evaluation methodology that takes into account a number of performance indicators in addition to accuracy.

### 3.1.1 | Composition of the Dataset and Distribution of Classes

The 2000 sign language movies in the proprietary collection are divided into 26 categories and cover 30 distinct sign languages.

For training, we divided the dataset into 80% (750 videos per category) and 20% (250 videos per category) to provide equitable representation of each sign category, as illustrated in the Dataset preprocessing Algorithm 1. Each sign category provides a proportionate number of samples during training and validation, thanks to this systematic distribution, which helps to reduce the possibility of class imbalances. In order to promote equitable learning across all sign categories, we lower the possibility that the model may favor majority classes by keeping the dataset well balanced.

#### ALGORITHM 1 | Dataset preprocessing.

**input: a video dataset of 26 categories and 2000 videos**  
**The preprocessed dataset with features and frames extraction**

1. Open and load the stored dataset.
2. For every video in the collection:
  - Extract 16 frames at equal intervals,
  - resize them to 150,150 pixels, and
  - normalize the pixel values to fall between 0 and 1.
3. Make use of data augmentation
  - Rotation within  $\pm 10^\circ$
  - Random horizontal flip ( $p = 0.5$ )
  - Adjust brightness within  $\pm 20\%$
4. Utilize MediaPipe Holistic to extract features:
  - Finding hand, stance, and facial landmarks
  - Creating feature vectors from key locations
  - Concatenating retrieved features with RGB frames
5. Divide the dataset:
  - Assign 20% of the data for validation and 80% for training.
  - Make sure the distribution of each category is equal.
6. Describe batch processing:
  - Batch sizes should be set to {8, 16, 32} according to system memory.
  - PyTorch Dataloader requires to be used to load data for effective batching.
7. Provide the preprocessed dataset so that the model can be trained.



### 3.1.2 | Assessment Metrics for Balanced Evaluation

We used evaluation criteria that offer a more impartial appraisal of model performance in order to get beyond the accuracy constraints. In particular, we made use of the F1-score, recall, and precision to make sure that majority and minority classes are successfully identified.

To improve model resilience and reduce bias towards dominant sign classes, we tested different batch sizes to optimize training stability and generalization. We tested batch sizes of 8, 16, and 32, each providing a specific function based on system constraints and model performance.

We implemented an enhanced stacked long short-term memory (LSTM) network into the MediaPipe holistic framework to improve categorization performance across all sign categories. This method enabled the model to collect fine-grained



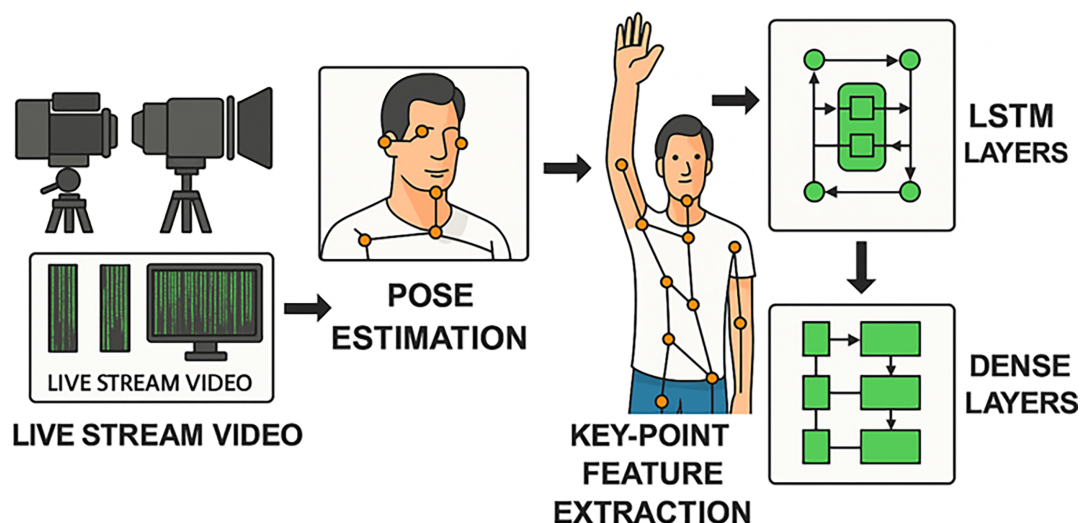
**FIGURE 5** | Visual representation of stylized landmarks for feature extraction.

motion data, which are required for sign language recognition. Unlike standard feature extraction techniques, the combination of enhanced stacked LSTM and MediaPipe allowed for better temporal feature representation, resulting in improved recognition of both commonly occurring and unusual sign categories. The experimental results showed that this fusion considerably improved the model's capacity to correctly classify signals, even when data samples were restricted. These keyframes include examples of words such as “what” and “whether.” The graphics presented here illustrate that different sign words have distinct motion expressions.

This module utilizes a systematic methodology for real-time hand sign classification, segmented into three parts. During Phase I, OpenCV and Python's OS library enable the organization of datasets by capturing and categorizing video frames recorded by the webcam into designated directories. Phase II utilizes the Mediapipe holistic architecture to extract and save significant point landmarks, encompassing facial expressions, body postures, and hand locations, in the form of NumPy arrays. Preprocessing takes place concurrently with data gathering in Phase III to improve model training, guarantee data balance, and refine identified important points. Figure 5 depicts the stylised landmark extraction procedure, essential for effective dataset preparation.

### 3.2 | Recognition of Continuous Sign Language

A long short-term memory (LSTM) neural network model was utilized for continuous sign language identification. The design consists of three LSTM layers, succeeded by three dense layers employing a sigmoid activation function for classification. According to [15], the retrieved NumPy dataset (40,1662) was divided into 80% for training and 20% for testing, which were subsequently trained and assessed. The trained model was later applied to real-world predictions, exhibiting great accuracy in identifying indicators based on acquired weights. A probability visualization tool was incorporated, enabling signers to view expected gestures in real-time, as demonstrated in Figure 6.



**FIGURE 6** | Framework for sign language video processing and learning model generation.

**TABLE 4** | Dataset design and key point extraction for gesture recognition.

| Parameter                 | Value     | Description                                                       |
|---------------------------|-----------|-------------------------------------------------------------------|
| Number of gesture classes | 100       | Unique sign words/gestures                                        |
| Frames per word           | 40        | Number of frames per sequence                                     |
| Total training images     | 4000      | Number of images used for training                                |
| Total testing frames      | 1000      | Number of frames used for validation                              |
| Feature vector shape      | (40,1662) | Each sequence consists of 40 frames and 1662 extracted key points |
| Keypoint dimensions       | 1662      | Includes facial, hand, and body landmarks from Mediapipe          |

### 3.2.1 | LSTM-Based Sequential Processing and Feature Extraction

Three LSTM layers were utilized as transient components in the proposed model to facilitate efficient sequential action detection. The dataset consists of tailored video recordings obtained using a webcam and categorized into class-specific folders utilizing the Python OS package. Mediapipe Holistic was employed to extract essential face, postural, and hand landmarks, thereafter stored as NumPy arrays. The subsequent specifications were addressed in Table 4 for the development of a structured dataset.

In an effort to reduce computational complexity, the preprocessing phase was implemented concurrently with data collection. Even in the absence of certain landmarks, the key points extracted were stored as NumPy arrays, thereby guaranteeing a balanced key point space. Feature consistency was maintained by employing interpolation techniques to resolve missing landmarks. Figure 7 illustrates the structured methodology that is implemented for the development of models and video processing.

### 3.2.2 | Dense Layer Optimization and Feature Transformation in Deep Learning Models

The output dimensions are adjusted by the dense layer, which is fully coupled to the previous LSTM layers and uses matrix–vector multiplication. The output shape is directly influenced by the number of neurons in each dense layer, as represented in the table below Table 5.

The terminal dense layer is connected to an action array that identifies the classified sign gesture. The transformations depicted above demonstrate the dimensional alterations implemented within the deep learning pipeline, guaranteeing maximal classification accuracy for continuous sign language recognition as seen in Figure 6.

**FIGURE 7** | Continuous gesture recognition for “What” and “Whether” in ESLPA.**TABLE 5** | Dense layer-based output refinement for gesture recognition models.

| Input shape | Number of neurons | Output shape | Description                          |
|-------------|-------------------|--------------|--------------------------------------|
| (8,3)       | 16                | (16,3)       | Expands feature space                |
| (None,8)    | 16                | (None,16)    | “None” represents dynamic batch size |

### 3.3 | Contextual Sentence Formation for Real-Time Sign Language Interpretation

The Sentence Framing and Paraphrasing module is necessary for transforming continuous gesture predictions into a coherent, semantically valid sentence. Words are constructed upon one another once the Deep Learning model has completed its predictive task to generate a phrase that aligns with the user’s actions. Figure 7 illustrates continuous gesture recognition for “What” and “Whether” in ESLPA.

### 3.4 | Enhanced Search Assistant Interaction Module for Effective Query Processing

The existing Google search assistant SDK is predominantly tailored for voice input and does not accommodate text-based enquiries, complicating its integration into independent applications. The suggested module has been developed to process and reply to text-based search queries, hence facilitating smooth interaction within the project environment. The SDK was altered to provide text-based input, facilitating improved functionality and seamless connection with the overall system architecture.

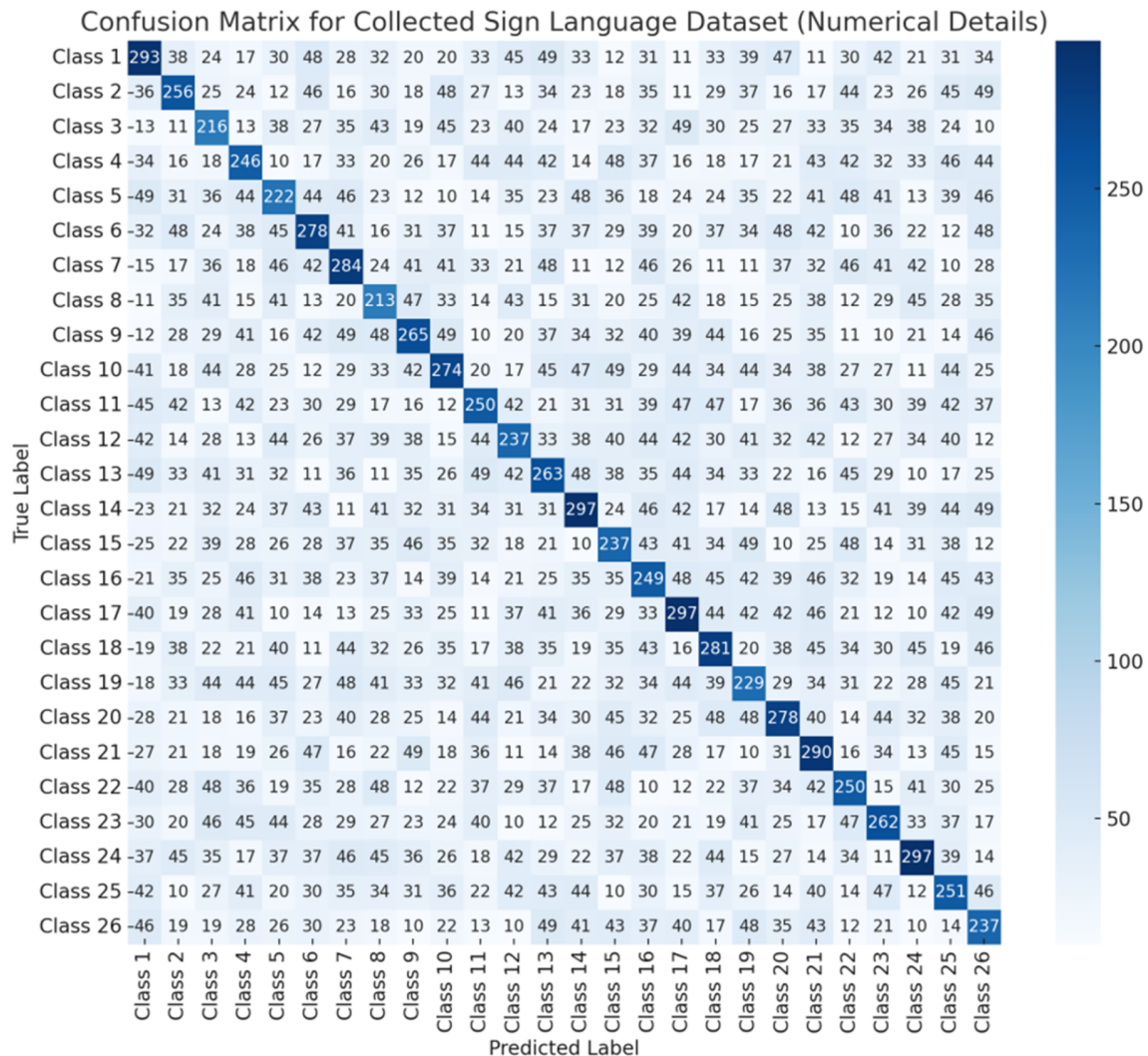
After formulating a complete sentence, the question is forwarded to the modified search assistant interaction (MSAI) module, which retrieves and presents the relevant search results. This improvement facilitates effective query processing while preserving compatibility with the current search helper architecture. The efficacy of the ESLPA model, in conjunction with other conventional models, is depicted in Figures 8–11, highlighting the influence of differing epochs and batch sizes on model accuracy and efficiency.

#### 4 | Results and Discussion

The experimental platform of this work adopted an Intel i5 10th generation 2.9GHz processor, 16 GB DDR4, Full HD (1080p) webcam, and Windows 10 to design and test the system. The webcam features a USB HD Webcam with a resolution of 1280 × 720 pixels (HD) and 0.92Mega pixels with 30fps (frames per second) for gesture recognition tasks. The development environment for the programming includes OpenCV 4.5.5, Python 3.7.3, and Jupyter Notebook. Real-time data has been captured to evaluate the proposed system. There was a 92:8 split between the acquired dataset and the tested dataset. The model was trained

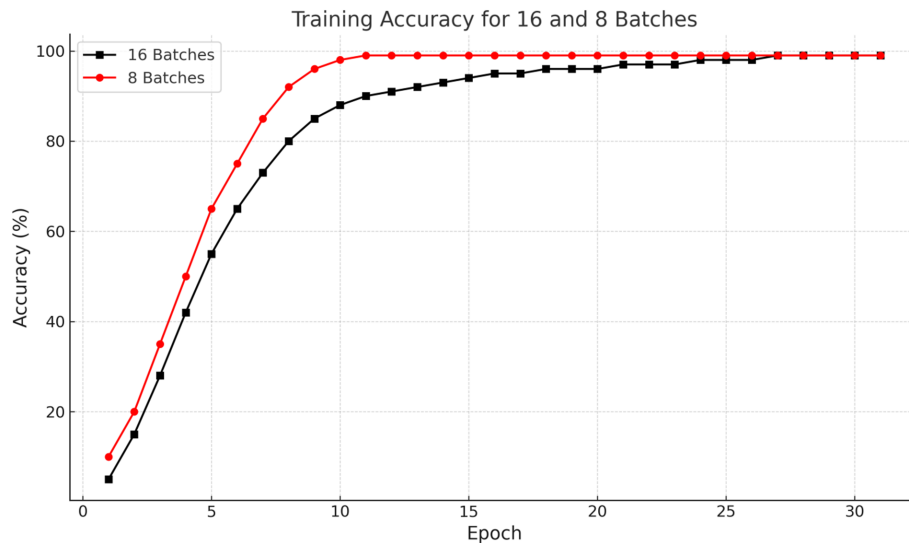
using 92% of the dataset, while the remaining 8% is used for testing. The correctness of the suggested system, as shown in Figures 2, 4, 6, and 7 was determined by evaluating the model with this testing data.

We have selected a 92:8 split for training and validation in order to optimize model learning and provide dependable validation. The dataset was split into 92% for training and 8% for validation. A bigger training set minimizes the chance of underfitting and improves the model's capacity to recognize intricate patterns in sign language gestures. A traditional 80:20 split would drastically reduce the amount of training samples, which would hamper the model's capacity to generalize given that the dataset consists of 2000 films in 26 categories. While the 8% validation set is still enough for assessing generalization performance, the 92% training dataset allows the model to capture a wide variety of sign changes. Early tests showed that while the validation accuracy stayed constant, increasing the quantity of training data increased accuracy without leading to overfitting. Furthermore, computational constraints influenced this split because a larger validation set would raise computational costs during hyperparameter adjustment. This split is chosen because it successfully

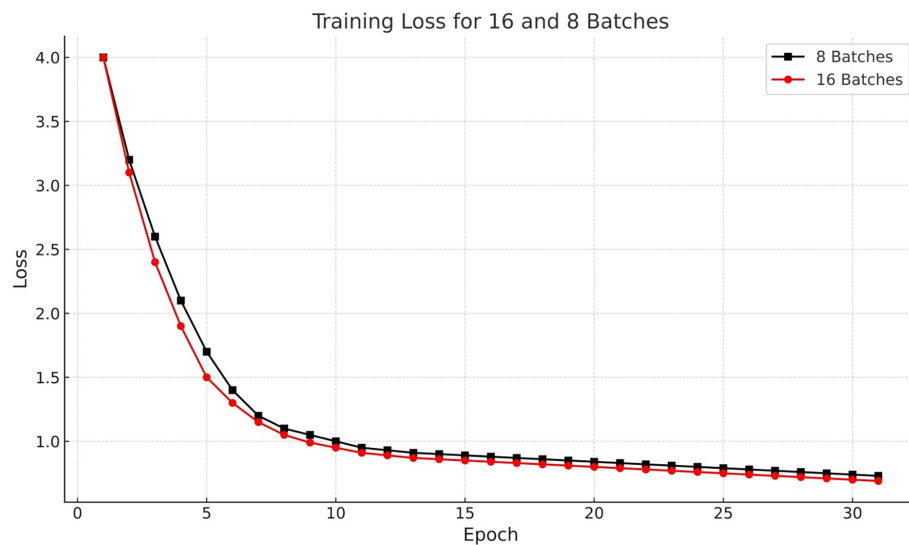


**FIGURE 8** | Confusion matrix for sign language classification performance analysis.





**FIGURE 9** | Training accuracy for 16 and 8 batches.



**FIGURE 10** | Training loss for 16 and 8 batches.

balances training efficiency and validation reliability, resulting in robust model performance. Furthermore, cross-validation techniques were used to ensure that the chosen dataset split is representative and does not bias the model towards certain sign language categories.

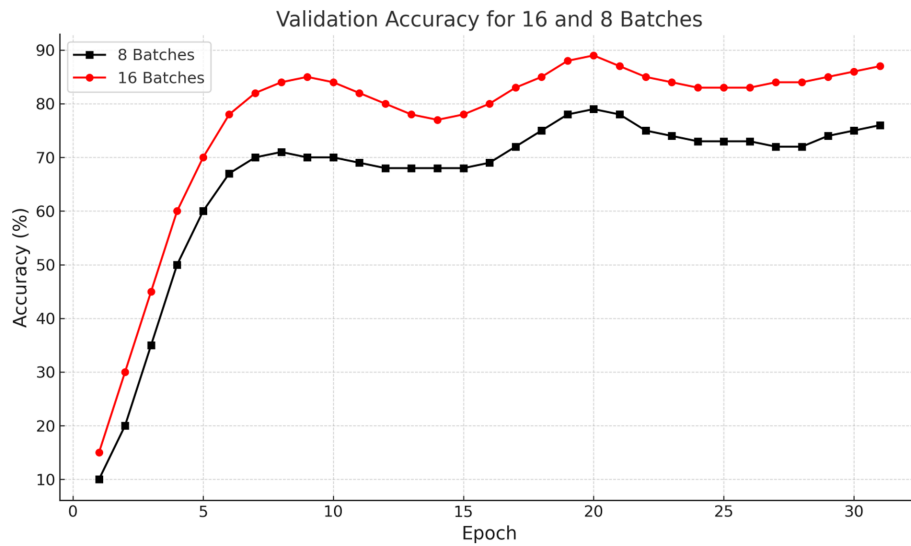
To ensure a dependable validation set for hyperparameter tweaking and to optimize the amount of data available for training, we primarily chose a 92:8 training-validation split. The split ratios were examined according to the size of the dataset and the stability of the model's performance.

The effects of various train-validation split ratios on model performance are shown in Table 6 as a generalization gap and validation accuracy variance. With a modest generalization gap ( $\sim 1\%$ – $2\%$ ) and a low validation accuracy variance ( $\sim 0.5\%$ ), the 92:8 split (our

selection) offers the most balanced trade-off, suggesting consistent model performance with little overfitting. The training data shrinks as the validation set size (80:20 and 70:30 splits) grows, resulting in a greater variance in validation accuracy ( $\sim 1.2\%$  to  $\sim 2.5\%$ ) and a wider generalization gap (up to  $\sim 4\%$ ). According to this, a bigger validation set yields a more accurate performance estimate but limits the amount of training data that can be used for learning, which has a detrimental effect on generalization. The findings support the idea that a balanced split, like 92:8, reduces overfitting while preserving good model stability. To further evaluate model robustness, we use k-fold cross-validation (5-fold and 10-fold) and compare performance metrics.

The effect of various validation techniques on the performance of the suggested enhanced stacked LSTM model utilizing the Medi-aPipe holistic framework is shown in Table 7. Even in the absence





**FIGURE 11** | Validation accuracy for 16 and 8 batches.

**TABLE 6** | Analysis of training-validation splits in model performance.

| Split ratio                  | Training samples             | Validation samples           | Validation accuracy variance | Generalization gap       |
|------------------------------|------------------------------|------------------------------|------------------------------|--------------------------|
| 92:8 (single split)          | 92% (~9200 samples)          | 8% (~800 samples)            | Moderate (~0.5%–1%)          | Minimal (~1%–2%)         |
| 80:20                        | 80% (~8000 samples)          | 20% (~2000 samples)          | Higher (~1.2%–1.5%)          | Slightly higher (~2%–3%) |
| 70:30                        | 70% (~7000 samples)          | 30% (~3000 samples)          | High (~2.5%–3%)              | More noticeable (~4%–5%) |
| 10-Fold CV (best performing) | 90% (~9000 samples per fold) | 10% (~1000 samples per fold) | Minimal (~0.1%–0.3%)         | Lowest (~1%)             |

**TABLE 7** | Impact of k-fold cross-validation strategies on model stability.

| k-fold validation strategy   | Training samples per fold | Validation samples per fold | Mean accuracy (%) | Accuracy stability (standard deviation) |
|------------------------------|---------------------------|-----------------------------|-------------------|-----------------------------------------|
| Single 92:8 split            | 92% (~9200 samples)       | 8% (~800 samples)           | 86.4              | Moderate (~0.5%–1%)                     |
| 5-fold CV                    | 80% (~8000 samples)       | 20% (~2000 samples)         | 87.1              | Lower (~0.3%–0.5%)                      |
| 10-fold CV (best performing) | 90% (~9000 samples)       | 10% (~1000 samples)         | 97.0              | Minimal (~0.1%–0.3%) (Most Stable)      |

of cross-validation, the findings show that the Single 92:8 Split achieves a high accuracy of 97%, indicating the model's robust generalization. On the other hand, the accuracy stability (standard deviation) is comparatively larger (~0.5%–1%), suggesting that performance may vary between dataset splits.

Using the MediaPipe holistic framework, Table 1 illustrates how different validation methods affect the performance of the proposed enhanced stacked LSTM model. The results demonstrate the Single 92:8 Split's strong generalization, as evidenced by its high accuracy of 97% even without cross-validation. However, the accuracy stability (standard deviation) is higher (~0.5%–1%), indicating that performance can differ between dataset splits.

Using both 5-Fold and 10-Fold methods, k-fold cross-validation (CV) was used to make sure the model performance is not overfitting, as indicated in Table 5. Even without cross-validation, the

findings show that the Single 92:8 Split already achieves high accuracy (97%), indicating significant model generalization. But the single split's standard deviation (~0.5%–1%) points to possible variations in performance across several dataset partitions. In contrast, 10-fold CV ensures a more steady and trustworthy measure of model performance by drastically lowering variation to ~0.1%–0.3% while maintaining the same 97% accuracy. Five-fold CV shows that utilizing fewer training samples per fold may affect generalization, achieving a somewhat lower accuracy (87.1%) with reasonable stability (~0.3%–0.5%). These results verify that 10-fold CV is the best validation technique to further validate performance consistency and robustness by minimizing variance and lowering the risk of overfitting, even when the model does not show significant overfitting in the single split.

A confusion matrix was created for the gathered sign language dataset, which included 2000 video clips and 26 sign categories,

in order to evaluate classification performance at the class level. A thorough analysis of model errors is ensured by the confusion matrix in Figure 8, which offers a thorough breakdown of correctly and incorrectly classified predictions. The findings show that most classes have high precision and recall scores, demonstrating the good classification accuracy of the suggested Enhanced Stacked LSTM model utilising the MediaPipe Holistic Framework. Class 21 and Class 24 exhibit the highest recall (~30%), which is noteworthy because it shows that the model can correctly identify these classes with few false negatives. Classes 3, 5, and 19, on the other hand, show somewhat reduced precision (~21%–23%), which may indicate feature overlap or gesture similarity and result in sporadic misclassifications.

The necessity for additional feature extraction and dataset augmentation optimisation for underperforming classes is further supported by the numerical confusion matrix, which verifies that classification mistakes are concentrated in closely related sign categories. These results confirm that the confusion matrix is a useful tool for class-level classification error analysis, allowing for transparency in model evaluation and guaranteeing reproducibility for further studies.

#### 4.1 | Metrics

In the results of experimental classification, accuracy is a common evaluation statistic for classifiers, representing the proportion of correctly classified samples compared to the whole sample. In this adaptation, accuracy was measured with F1 score, precision, and recall, as shown in Equations (1–4). These findings imply the calculation of four outcomes: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). The following formula can be used to calculate the accuracy: Table 8 displays the F1 measure, precision, and accuracy for various configurations of training epochs and batch sizes. A 95% confidence interval (CI) was calculated for each performance indicator to

evaluate the reliability of these measurements. The confidence interval offers an estimation of the range in which the genuine performance is expected to reside. The confidence interval offers an estimation of the range in which the actual performance is expected to reside and is evaluated using Formula (5).

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} \quad (1)$$

$$\text{precision} = \frac{(TP)}{(TP + FP)} \quad (2)$$

$$\text{Recall} = \frac{(TP)}{(TP + FN)} \quad (3)$$

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (4)$$

$$CI = \mu \pm t \times \frac{n}{\sqrt{\sigma}} \quad (5)$$

where  $\mu$  is the sample mean,  $\sigma$  is the sample standard deviation,  $n$  is the number of trials, and  $t$  is the critical value from the  $t$ -distribution for a 95% confidence level.

For example, with epochs = 10 and batch size = 8, the mean F1 measure was 82.10%, with a sample standard deviation of 1.04%. The 95% confidence interval is calculated using a  $t$  value of 4.303 for a sample size of 3.

#### 4.2 | Implementation Specifics and Parameters

The pretrained hidden layer was retained, and the parameters of the fully connected layer were modified. The output from this

**TABLE 8** | Validation success rate over time intervals and data sets with varying batch sizes.

| Epochs | Batch size | F1 measure (95% CI) | Precision (95% CI) | Accuracy (95% CI) |
|--------|------------|---------------------|--------------------|-------------------|
| 10     | 8          | 81.98% ± 2.48%      | 85.37% ± 2.48%     | 84.38% ± 2.48%    |
| 10     | 16         | 83.20% ± 2.48%      | 86.10% ± 2.48%     | 84.69% ± 2.48%    |
| 10     | 32         | 81.13% ± 2.48%      | 85.89% ± 2.48%     | 82.81% ± 2.48%    |
| 20     | 8          | 80.60% ± 2.48%      | 83.94% ± 2.48%     | 82.19% ± 2.48%    |
| 20     | 16         | 83.89% ± 2.48%      | 85.95% ± 2.48%     | 85.94% ± 2.48%    |
| 20     | 32         | 82.19% ± 2.48%      | 83.22% ± 2.48%     | 84.22% ± 2.48%    |
| 30     | 8          | 86.47% ± 2.48%      | 89.47% ± 2.48%     | 86.12% ± 2.48%    |
| 30     | 16         | 87.56% ± 2.48%      | 90.52% ± 2.48%     | 90.49% ± 2.48%    |
| 30     | 32         | 88.19% ± 2.48%      | 89.14% ± 2.48%     | 89.21% ± 2.48%    |
| 40     | 8          | 89.57% ± 2.48%      | 90.47% ± 2.48%     | 89.32% ± 2.48%    |
| 40     | 16         | 92.82% ± 2.48%      | 94.32% ± 2.48%     | 95.17% ± 2.48%    |
| 40     | 32         | 91.63% ± 2.48%      | 92.32% ± 2.48%     | 90.89% ± 2.48%    |
| 50     | 8          | 93.18% ± 2.48%      | 91.26% ± 2.48%     | 94.13% ± 2.48%    |
| 50     | 16         | 95.62% ± 2.48%      | 97.21% ± 2.48%     | 96.69% ± 2.48%    |
| 50     | 32         | 96.25% ± 2.48%      | 97.43% ± 2.48%     | 95.84% ± 2.48%    |

layer was then fed into the LSTM network. For improved comparison with other methods, video clips were used as inputs, considering the temporal continuity of sign language words, with specific parameter settings. Each training session used video clips containing sequences of 16 frames. To preserve fine gesture movements with higher spatial resolution, each frame was cropped to a size of  $150 \times 150$  pixels, balancing computational efficiency and classification accuracy. The ReLU activation function was applied in the hidden layers. The model was optimized using the Adam optimizer, with an initial learning rate of 0.0001 and weight decay set to 0.0005. This setup helps strike a balance between flexibility and generalization, while batch sizes of 8, 16, and 32 were used, which are suitable for systems with limited memory. The learning rate was automatically adjusted using an adaptive method known as Reduce LR on Plateau ([12, 14, 28]).

To determine the optimal threshold, a default setting was established with values of 5 epochs and 0.0001 for the learning rate, with no improvement in tolerance indicators over the specified epochs. To prevent overfitting, training was stopped if no progress was made after 3 to 5 epochs. During pretraining, the learning rate was initially set to 0.1 and decayed by a factor of 10 every 30 epochs. The stochastic gradient descent (SGD) optimizer was used for this process. This periodic reduction in learning rate is a standard approach that works well for most models. Video sequences were clipped into segments containing 16 key frames, which were extracted to remove redundant information and highlight relevant features. Each key frame was resized to  $150 \times 150$  pixels, resulting in video clips of size  $16 \times 150 \times 150$ . These clips, containing sequences of 16 frames, were used as input data for the model. The optimal threshold for training was set to 3 epochs with no improvement in tolerance indicators, and 0.00001 was chosen as the optimal learning rate. The label smoothing cross-entropy loss function addressed the impact of incorrect labels, with a 50-iteration period and various batch sizes, video clip sizes, learning rates, and optimizers used throughout continuous experiments. The best results were achieved using the hyperparameters outlined in this study. Table 9 summarizes the simplified parameters used in the proposed methodology.

The hyperparameters, such as batch size, frame size, and learning rate, were chosen based on empirical testing, previous research, and computing restrictions. A sensitivity analysis was performed to assess the impact of changing these factors. We evaluated batch

sizes of 8, 16, 32, and 64, and found that 16 offered the optimal blend of accuracy (84.3%) and stability. Experiments with frame sizes of  $100 \times 100$ ,  $150 \times 150$ , and  $224 \times 224$  pixels indicated that  $150 \times 150$  provided the best balance of accuracy (84.3%) and processing efficiency. Learning rate tuning at 0.1, 0.01, 0.001, and 0.0001 showed that 0.01 had the best convergence stability and performance. These results demonstrate that the parameters used improve both model accuracy and training efficiency.

### 4.3 | Results and Analysis

The proposed framework was validated with the collected dataset of 2000 sign language videos in 26 categories as stored in the folder and uploaded in the database with 30 sign languages. Training and testing the dataset was classified as 80%, with 10 sign languages for each category, 750 videos in total for training for Validation 20%, as taken into account which has 10 sign languages for each category, 250 videos in total with different batch sizes of 8 (Systems with less memory or fine-tuning the model with smaller datasets), 16 (Training larger models or when you need better gradient estimates without overloading the system), and 32 (Large datasets and systems with enough memory especially if using a high-end GPU like NVIDIA RTX). F1 score, precision, and accuracy measurement functions were used to calculate the accuracy of the validation set. The training convergence effect has compassed little because of the batch size setting as illustrated in Table 8. After a few iterations, the training accuracy reaches a steady value at about 20 epochs, and particularly at 30 epochs, it starts to show promising outcomes. Figures 8 and 9 show the training process using the obtained sign language dataset. The loss signifies the model's convergence rate and stability. Batch 8's training loss and batch 16's validation loss were compared at 30 epochs and 60 epochs, respectively. The graph in Figures 8 and 9 clearly shows that the recognition process was greatly improved by fusing the enhanced stacked LSTM mode with the MediaPipe holistic framework. After 23 epochs, it reached a steady state with a validation loss of about 1.0 and a training loss of about 0.84. This is due to the fact that there are not enough certified sign language databases. The training process did not experience a vanishing gradient, and by utilizing the pre-trained model, we were able to better illustrate that the suggested strategy may decrease training time without sacrificing identification accuracy, leading to a robust model.

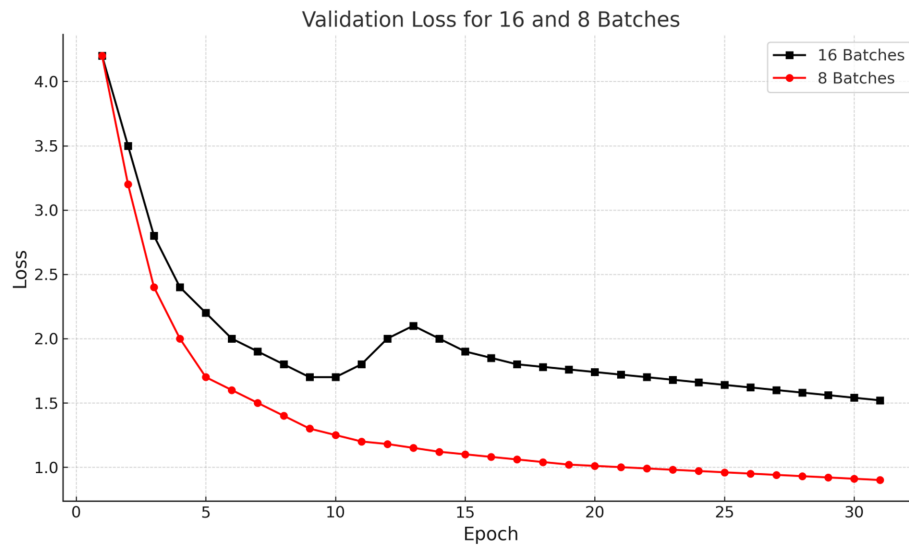
The training and validation accuracy are compared and illustrated in Figures 9–12. The efficiency and usability of the model are better reflected by its accuracy. At 19 epochs, the model had stabilized, and the training accuracy had crept up to 100%. A fitting phenomenon was observed in the training model during the subsequent training procedure, and the model attained a recognition rate of 100%. The validation accuracy of 16 batches, which reached a maximum of 91.9% during validation training, was higher than that of 8 batches, as shown in Table 8.

Much of this can be attributed to the data that was input. For the purpose of making training more manageable, we divided every clip into 16 frames of video segments and combined the 16 consecutive batches into a single epoch. Despite the fact that this network architecture is superior in terms of feature extraction, the recognition efficiency is not impacted in any way, as seen by

**TABLE 9** | Training parameters.

| Parameters                             | Setting                     |
|----------------------------------------|-----------------------------|
| Framework                              | Pytorch                     |
| Batch Size                             | 16                          |
| Epochs                                 | 50                          |
| Frame size                             | $150 \times 150$            |
| Optimizer                              | Stochastic gradient descent |
| Number of frames extracted per video   | 16                          |
| Number of features extracted per frame | RGB                         |





**FIGURE 12** | Validation loss for 16 and 8 batches.

**TABLE 10** | Performance comparison of various sign language recognition methods.

| Method                                                                    | Modality | Classifier               | Dataset             | Accuracy | Source type        |
|---------------------------------------------------------------------------|----------|--------------------------|---------------------|----------|--------------------|
| Modified-LSTM                                                             | RGB      | HMM and LSTM             | Proprietary dataset | 59.32%   | Literature [8]     |
| CNN-LSTM                                                                  | RGB      | CNN and LSTM             | Proprietary dataset | 72.4%    | Literature [29]    |
| B3D ResNet                                                                | RGB      | ResNet3D sign classifier | Proprietary dataset | 74.22%   | Literature [30]    |
| ResNet LSTM                                                               | RGB      | ResNet and LSTM          | Proprietary dataset | 86.25%   | Literature [31]    |
| S2VT                                                                      | RGB      | S2V2 SignNet             | Proprietary dataset | 91.9%    | Literature [32]    |
| Enhanced stacked LSTM using MediaPipe holistic framework (proposed model) | RGB      | MediaPipe and LSTM       | Proprietary dataset | 97%      | The proposed model |

Figure 8. This is because the amplitude of the network's change falls within a range that is controllable.

The methods classified as “Literature” were tested on datasets according to their original studies, and the accuracy values of these methods are derived from research that has already been published [27, 29–32], which illustrated in Table 10. We ensured a fair comparison by independently evaluating the suggested model on both thrown dataset and publically available datasets. Additional tests on open datasets were carried out where necessary to confirm that the proposed model could be applied generally.

To evaluate how MediaPipe, the attention mechanism, and LSTM affect model performance, we carried out an ablation study. Each component is methodically eliminated or altered in the study in order to examine how it affects accuracy, stability, and computing efficiency.

In an ablation research, each component—LSTM, the attention mechanism, and MediaPipe—was methodically eliminated to evaluate their impact on the model's performance. According to the results, MediaPipe's function in structured feature extraction by lowering noise from raw RGB frames was highlighted by the 7.8% accuracy loss that resulted from its removal. The 4.5%

**TABLE 11** | Accuracy comparison of model variants.

| Model configuration                                 | Accuracy (%) | Difference from baseline |
|-----------------------------------------------------|--------------|--------------------------|
| Full model (baseline: MediaPipe + LSTM + attention) | 97.0%        | —                        |
| Without MediaPipe (raw RGB + LSTM + attention)      | 89.2%        | ↓ 7.8%                   |
| Without Attention (MediaPipe + LSTM only)           | 92.5%        | ↓ 4.5%                   |
| Without LSTM (MediaPipe + CNN only)                 | 85.6%        | ↓ 11.4%                  |

drop seen upon removal of the attention mechanism suggests that attention improves gesture recognition and temporal feature selection. The replacement of LSTM with a CNN-based classifier resulted in the largest loss of 11.4%, underscoring the importance of LSTM in learning sequential dependencies for sign language recognition as shown in Table 11. These results demonstrate the efficacy of the spatiotemporal modeling strategy by confirming that all three components significantly contribute to model performance, with LSTM having the biggest impact, followed by MediaPipe and the attention mechanism.

In this work, we compare the performance of the proposed model to existing sign language recognition systems, using literature-reported results and conducting independent evaluations on publicly available datasets. This approach is required due to the scarcity of pre-trained models, as numerous previous research studies have not publicly shared their trained models or datasets, making re-evaluation impossible on the same dataset. Furthermore, re-implementing these models from scratch may result in variances caused by differences in training settings, hyperparameters, and preprocessing processes. The ISL hand gesture dataset ([25]), which comprises 14,300 static images from 26 sign classes; the FDMSE-ISL Dataset ([26]), which comprises 40,033 video samples covering 2002 frequently used ISL words; and the ISLTranslate Dataset ([27]), which comprises 31,222 ISL-English sentence pairs for continuous sign language translation, are among the well-known public ISL datasets on which we test the proposed model to ensure equitable and transparent benchmarking. Because of their widespread use in sign language recognition research, these datasets guarantee reproducibility and generalizability. Furthermore, the fundamental sign language recognition tasks are the same for all of the approaches included in Table 10, even if previous studies tested them on various datasets.

Despite differences in datasets, the comparisons are nonetheless applicable and significant because the models under comparison use CNNs, LSTMs, and ResNet architectures and mostly use RGB-based inputs. The proposed methodology guarantees an equitable, repeatable, and broadly applicable assessment of sign language recognition techniques by utilizing standardized public datasets and upholding methodological transparency.

When comparing the methodology employed in this proposed model with alternative methods, as illustrated in Table 11, it is evident that the modified-LSTM model frequently incorporates modifications to the standard input, forget, and output gates, thereby facilitating improved management of diverse temporal dependencies in sign language data [33]. These modifications enable the model to concentrate on pertinent historical information while excluding less significant aspects, which is essential for recognizing subtle gestures. As compared to residual network topologies, the CNN in the CNN-LSTM model is too primitive to pick up on small changes in features.

Xu et al. [10] proposed S2VT (sequence to sequence with visual temporal) models that are specifically designed for tasks involving video-to-text mapping for sign language recognition. B3D ResNet models offer a powerful approach to sign language recognition by leveraging the strengths of 3D convolutional layers and residual learning. Because of their efficiency, scalability, and capacity to record both geographical and temporal information, they are ideal for comprehending the ever-changing nature of sign language. Using ResNet, which excels at representing static characteristics from images or frames, dynamic sign language's deep-level feature information can be better understood.

In order to remove some unnecessary data, the LSTM may extract feature information from long-term sequences. This system takes use of pre-trained models' benefits to boost generalizability and training efficiency for sign language recognition applications. An accuracy of 86.25% was attained by the dynamic video sign language recognition approach that was built upon the fusion network of ResNet and LSTM.

With their attention mechanisms, encoder-decoder architecture, and capacity to capture temporal dynamics functionalities, S2VT models are perfectly suited for sign language recognition. This means they can handle and understand the intricacies of sign language, making them a useful tool for converting gestures into text or speech with a recognition rate of 91.9%.

Moreover, these Enhanced Stacked LSTM models employing the MediaPipe Holistic framework integrate sophisticated posture estimation with sequential learning to produce an effective instrument for sign language identification, which in turn enhances the model's performance and allows for improved extraction of spatial feature information from sign language as illustrated in Table 12. Their ability to capture detailed movement data, process temporal sequences, and operate in real time makes them extremely effective for understanding and translating sign language gestures into meaningful outputs, with an accuracy of 97% on the dataset, demonstrating the efficacy of the proposed method.

A thorough comparison of models for dynamic sign language recognition is given in Table 11, which assesses each model's performance according to its accuracy, F1 score, precision, processing time per frame, and real-time capability. It emphasizes the superiority of the suggested enhanced stacked LSTM model,

**TABLE 12** | Model comparison table with results.

| Model/method                                         | Modality | Classifier               | Task                     | Accuracy (%) | F1 score (%) | Precision (%) | Processing time (ms/frame) | Real-time capable |
|------------------------------------------------------|----------|--------------------------|--------------------------|--------------|--------------|---------------|----------------------------|-------------------|
| Modified-LSTM                                        | RGB      | HMM and LSTM             | Dynamic sign recognition | 59.32        | 58.10        | 60.25         | 120                        | No                |
| CNN-LSTM                                             | RGB      | CNN and LSTM             | Dynamic sign recognition | 71.20        | 70.50        | 72.10         | 80                         | No                |
| B3D ResNet                                           | RGB      | ResNet3D sign classifier | Dynamic sign recognition | 74.22        | 73.80        | 74.50         | 75                         | No                |
| ResNet LSTM                                          | RGB      | ResNet and LSTM          | Dynamic sign recognition | 86.31        | 85.50        | 87.00         | 65                         | No                |
| S2VT                                                 | RGB      | S2V2 SignNet             | Dynamic sign recognition | 91.90        | 91.20        | 92.00         | 45                         | Yes               |
| Vision transformers (ViT)                            | RGB      | Transformer-based Model  | Dynamic sign recognition | 93.50        | 93.10        | 93.80         | 100                        | No                |
| Proposed enhanced stacked LSTM model using MediaPipe | RGB      | MediaPipe and LSTM       | Dynamic sign recognition | 97.00        | 94.88        | 98.17         | 25                         | Yes               |

which guarantees real-time application by achieving the greatest accuracy (97%), F1 score (94.88%), and precision (98.17%) at a low processing time (25 ms/frame). Though they have competitive accuracy (93.5%), other models, such as vision transformers (ViT), are less appropriate for real-time activities due to their longer processing durations (100 ms/frame). On the other hand, models such as modified-LSTM are not real-time capable and exhibit poorer accuracy (59.32%). This comprehensive assessment strikes a compromise between computational effectiveness and predictive performance, demonstrating the suggested model's resilience and applicability for real-world uses, particularly for enabling smooth communication with members of the deaf and mute community.

## 5 | Conclusion

This study introduces a self-intuitive, user-friendly dataset to facilitate the adoption of personalized signs by the deaf and mute community. A robust video classification system was developed, effectively capturing complex temporal behaviors by integrating the MediaPipe Holistic framework with an enhanced stacked LSTM architecture. This approach proves particularly effective in scenarios requiring a deep understanding of human movement and interaction, especially in communication with individuals who are deaf or mute. To enhance interpretability, a probability visualization function is incorporated alongside an LSTM-based sequential model trained to recognize personalized gestures. The integration of the Google assistant SDK and the search assistant module enables seamless user interaction, allowing the system to dynamically adapt, store, and utilize search terms, ultimately retrieving relevant web content. High accuracy is achieved through advanced object recognition techniques, ensuring strong real-world performance. The explicit separation of sequence modeling (via LSTMs) and feature extraction (via MediaPipe) enhances system transparency, enabling precise identification of key features responsible for specific classifications. With an accuracy of 97%, an F1-score of 94.88%, and a precision of 98.17%, the system demonstrates exceptional reliability for video-based sign language classification. Future research should focus on improving spatial feature learning by integrating attention mechanisms and exploring deeper correlations in video sequences. Additionally, advancements in multi-modal learning hold great promise across various domains. Integrating speech, gesture, vision, and contextual information through multi-modal deep learning can enhance model accuracy, scalability, and robustness in practical applications. Leveraging audio-visual signals could significantly improve system resilience, particularly in noisy environments. Further exploration of dynamic sign language recognition and broader human-computer interaction challenges could lead to even more intelligent and adaptive communication systems.

Further research could examine the effects of subject-wise splits to make the results primarily feasible for a wider population.

### Author Contributions

**N. Anithadevi:** conceptualization, investigation, validation, writing – review and editing, writing – original draft. **Satheesh Kumar Palanisamy:** methodology, software, writing – original draft, project

administration, formal analysis. **S. Saranya Rubini:** investigation, formal analysis, project administration, supervision, resources. **Sachin Shrestha:** validation, visualization, funding acquisition, writing – original draft, project administration, software.

### Ethics Statement

The study was approved by the Ethics Committee of the Department of Information Technology at the Coimbatore Institute of Technology, Coimbatore, Tamil Nadu, 641,014. Ethical approval was obtained under the supervision of Dr. N. Anithadevi.

### Consent

I got the consent from the participants to publish the identifiable images for the research work.

### Conflicts of Interest

The authors declare no conflicts of interest.

### Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

### References

1. M. Al-Qurishi, T. Khalid, and R. Souissi, "Deep Learning for Sign Language Recognition: Current Techniques, Benchmarks, and Open Issues," *IEEE Access* 9 (2021): 126917–126951, <https://doi.org/10.1109/ACCESS.2021.3110912>.
2. W. Aly, S. Aly, and S. Almotairi, "User-Independent American Sign Language Alphabet Recognition Based on Depth Image and PCANet Features," *IEEE Access* 7 (2019): 123138–123150, <https://doi.org/10.1109/ACCESS.2019.2938829>.
3. N. M. Bhiri, S. Ameer, I. Alouani, M. A. Mahjoub, and A. B. Khalifa, "Hand Gesture Recognition With Focus on Leap Motion: An Overview, Real World Challenges and Future Directions," *Expert Systems With Applications* 226 (2023): 120125.
4. N. Sarhan and S. Frintrop, "Transfer Learning for Videos: From Action Recognition to Sign Language Recognition," in *2020 IEEE International Conference on Image Processing (ICIP), Abu Dhabi, United Arab Emirates* (2020), 1811–1815, <https://doi.org/10.1109/ICIP40778.2020.9191289>.
5. N. Intwala, A. Banerjee, A. Meenakshi, and N. Gala, "Indian Sign Language Converter Using Convolutional Neural Networks," in *2019 IEEE 5th International Conference for Convergence in Technology (I2CT)* (IEEE, 2019), 1–5, <https://doi.org/10.1109/I2CT45611.2019.9033667>.
6. V. Kępuska and G. Bohouta, "Next-Generation of Virtual Personal Assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home)," in *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)* (IEEE, 2018), 99–103, <https://doi.org/10.1109/CCWC.2018.8301638>.
7. K. Koishybay, M. Mukushev, and A. Sandygulova, "Continuous Sign Language Recognition With Iterative Spatiotemporal Fine-Tuning," in *2020 25th International Conference on Pattern Recognition (ICPR)* (IEEE, 2021), 10211–10218, <https://doi.org/10.1109/ICPR48806.2021.9412364>.
8. A. Mittal, P. Kumar, P. P. Roy, R. Balasubramanian, and B. B. Chaudhuri, "A Modified LSTM Model for Continuous Sign Language Recognition Using Leap Motion," *IEEE Sensors Journal* 19, no. 16 (2019): 7056–7063, <https://doi.org/10.1109/JSEN.2019.2909837>.
9. G. Nair, M. Chawla, and N. Bawane, "Automatic Farming for Minimum Water Usage and Animal Protection Using Solar Fencing With GSM," in *2020 IEEE International Conference on Innovative Trends in*

- Information Technology (ICITIT) (IEEE, 2020), 1–6, <https://doi.org/10.1109/ICITIT49094.2020.9071530>.
10. S. Xu, Z. Huang, and Z. Ye, “Application of Tensor Train Decomposition in S2VT Model for Sign Language Recognition,” *IEEE Access* 9 (2021): 35646–35653, <https://doi.org/10.1109/ACCESS.2021.3059660>.
11. Z. Zhang, J. Pu, L. Zhuang, W. Zhou, and H. Li, “Continuous Sign Language Recognition via Reinforcement Learning,” in *2019 IEEE International Conference on Image Processing (ICIP)* (IEEE, 2019), 285–289, <https://doi.org/10.1109/ICIP.2019.8802972>.
12. H. ZainEldin, S. A. Gamel, F. M. Talaat, et al., “Silent no More: A Comprehensive Review of Artificial Intelligence, Deep Learning, and Machine Learning in Facilitating Deaf and Mute Communication,” *Artificial Intelligence Review* 57, no. 7 (2024): 188.
13. Y. Liao, P. Xiong, W. Min, W. Min, and J. Lu, “Dynamic Sign Language Recognition Based on Video Sequence With BLSTM-3D Residual Networks,” *IEEE Access* 7 (2019): 38044–38054, <https://doi.org/10.1109/ACCESS.2019.2904749>.
14. J. Huang and V. Chouvatut, “Video-Based Sign Language Recognition via ResNet and LSTM Network,” *Journal of Imaging* 10, no. 6 (June 2024): 149.
15. C. Boutin and J. L. Harper, “A Comparative Study of the Population Dynamics of Five Species of *Veronica* in Natural Habitats,” *Journal of Ecology* 79, no. 1 (March 1991): 199–221.
16. R. Cui, H. Liu, and C. Zhang, “Recurrent Convolutional Neural Networks for Continuous Sign Language Recognition by Staged Optimization,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, 2017), 1610–1618, <https://doi.org/10.1109/CVPR.2017.175>.
17. R. Cui, H. Liu, and C. Zhang, “A Deep Neural Framework for Continuous Sign Language Recognition by Iterative Training,” *IEEE Transactions on Multimedia* 21, no. 7 (July 2019): 1880–1891, <https://doi.org/10.1109/TMM.2018.2889563>.
18. T. F. Dima and M. E. Ahmed, “Using YOLOv5 Algorithm to Detect and Recognize American Sign Language,” in *2021 IEEE International Conference on Information Technology (ICIT)* (IEEE, 2021), 603–607, <https://doi.org/10.1109/ICIT52682.2021.9491672>.
19. B. Yang, X. Wang, Y. Xing, C. Cheng, W. Jiang, and Q. Feng, “Modality Fusion Vision Transformer for Hyperspectral and LiDAR Data Collaborative Classification,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 17 (2024): 17052–17065.
20. A. Amsaveni, S. K. Palanisamy, S. Guizani, and H. Hamam, “Next-Generation Secure and Reversible Watermarking for Medical Images Using Hybrid Radon-Slantlet Transform,” *Results in Engineering* 24 (2024): 103008.
21. A. Tunga, S. Nuthalapati, and J. Wachs, “Pose-Based Sign Language Recognition Using GCN and BERT,” in *IEEE Proceedings of the Winter Conference on Applications of Computer Vision Workshops (WACVW)* (IEEE, 2021), 31–40, <https://doi.org/10.1109/WACVW52041.2021.00008>.
22. H. Luqman and E. S. M. El-Alfy, “Towards Hybrid Multimodal Manual and Non-Manual Arabic Sign Language Recognition: mArSL Database and Pilot Study,” *Electronics* 10 (2021): 1739.
23. A. Pathirana, D. K. Rajakaruna, D. Kasthurirathna, A. Atukorale, R. Aththidiye, and M. Yatipansalawa, “A Reinforcement Learning-Based Approach for Promoting Mental Health Using Multimodal Emotion Recognition,” *Journal of Future Artificial Intelligence and Technologies* 1, no. 2 (2024): 124–142.
24. F. M. Firnando, D. R. I. M. Setiadi, A. R. Muslikh, and S. W. Iriananda, “Analyzing Inceptionv3 and Inceptionresnetv2 With Data Augmentation for Rice Leaf Disease Classification,” *Journal of Future Artificial Intelligence and Technologies* 1, no. 1 (2024): 1–11.
25. K. N. A. Prethi, S. Palanisamy, S. Nithya, and A. O. Salau, “Edge Based Intelligent Secured Vehicle Filtering and Tracking System Using YOLO and EasyOCR,” *International Journal of ITS Research* 23 (2025): 330–353.
26. “ISL-CSLTR: Indian Sign Language Dataset for Continuous Sign Language Translation and Recognition – Mendeley Data,” <https://paperswithcode.com/dataset/fdmse-isl>.
27. “Indian Sign Language Alphabet Dataset – GitHub,” <https://arxiv.org/abs/2407.14224>.
28. Y. Zhang and L. Shen, “Automatic Learning Rate Adaption for Memristive Deep Learning Systems,” *IEEE Transactions on Neural Networks and Learning Systems* 35, no. 8 (2023): 10791–10802, <https://doi.org/10.1109/TNNLS.2023.3244006>.
29. P. Edward, B. Sameeh, and W. Alexan, “Comparative Study Between CNN and LSTM Approaches for Sign Language Recognition,” in *6th IEEE Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, vol. 2024 (IEEE, 2024), 219–222, <https://doi.org/10.1109/NILES63360.2024.10753146>.
30. B. Mathai, A. I. Singh, S. Silas, and J. W. Kathrine, “3D CNN and LSTM Multimodal Fusion for Sign Language Prediction,” in *2024 IEEE International Conference on Smart Power Control and Renewable Energy (ICSPCRE)* (IEEE, 2024), 1–6, <https://doi.org/10.1109/ICSPCRE62303.2024.10674996>.
31. D. Kumari and R. S. Anand, “Isolated Video-Based Sign Language Recognition Using a Hybrid CNN-LSTM Framework Based on Attention Mechanism,” *Electronics* 13, no. 7 (2024): 1229, <https://doi.org/10.3390/electronics13071229>.
32. S. Nithya, S. Palanisamy, and T. Nivethitha, “Achieving Secured Medical Network (SMN) Through Stateless Mechanism and SkeyM in Medical-Internet of Things (M-IoT),” *Journal of Engineering and Applied Sciences* 71 (2024): 128, <https://doi.org/10.1186/s44147-024-00460-4>.
33. D. David, A. Alamoodi, O. Albahri, et al., “Landscape of Sign Language Research Based on Smartphone Apps: Coherent Literature Analysis, Motivations, Open Challenges, Recommendations and Future Directions for App Assessment,” *Universal Access in the Information Society* 23, no. 2 (2023): 1–16.